Module 11 Reflection
Course: IS601
Name: Kush Anil Yarwar

Overview

Module 11 built directly on the foundation I set up in Module 10. While Module 10 focused on building a secure User model with password hashing and CI/CD, Module 11 shifted my attention to data modeling, input validation, and software design patterns.

This module required me to:

- Define a SQLAlchemy Calculation model with proper field types
- Create Pydantic schemas with custom validators
- Implement a factory design pattern for arithmetic operations
- Write comprehensive unit and integration tests for the new functionality

Each of these pushed me to think more carefully about how data flows through an application—from the request body, through validation, into the database, and back out as a response.

Defining the Calculation Model

The Calculation model introduced a few new challenges compared to the User model.

Fields I defined:

- a and b — Float operands
- type — String storing the operation type
- result — Float computed at creation time and stored in the database
- timestamp — Auto-generated using server_default=func.now()
- user_id — Foreign key referencing the users table with cascade delete

Key Decision — Store or Compute:

I chose to store the result in the database rather than computing it on demand. This means:

- Queries are faster since no calculation is needed at read time
- Historical results are preserved even if the logic changes

The user_id foreign key also meant I had to ensure referential integrity. If a user is deleted, all their calculations are automatically removed using cascade="all, delete".

OperationType Enum

Instead of accepting any string for the type field, I used Python's Enum class to restrict valid values:

- Add
- Sub
- Multiply
- Divide

By making OperationType inherit from both str and Enum, Pydantic automatically validates the field and rejects anything outside these four values with a 422 error. This eliminates an entire category of bad input without writing extra validation logic.

Factory Pattern

The factory pattern was the most interesting new concept in this module.

How it works:

- Each operation type has its own class (AddOperation, SubOperation, MultiplyOperation, DivideOperation)
- Each class implements a compute(a, b) method
- CalculationFactory holds a registry mapping each OperationType to the correct class
- Calling CalculationFactory.compute(type, a, b) selects and runs the right operation

Why this matters:

- Adding a new operation in the future only requires adding a new class and one registry entry
- The route handler does not need to know anything about how operations work
- Each operation class can be tested independently

This creates a clean separation of concerns:

- The factory handles which operation
- The operation class handles how to compute
- The route handler handles what to store

Pydantic Schemas and Custom Validation

Module 11 required more advanced Pydantic usage than Module 10.

CalculationCreate:

- Uses OperationType enum to validate the type field
- Uses @model_validator(mode="after") to check for division by zero after all fields are parsed

Key Insight:

The divide-by-zero check had to be a model-level validator (not a field-level one) because it depends on two fields together—type and b. A field validator only sees one field at a time.

CalculationRead:

- Returns all fields including the computed result
- Uses model_config = {"from_attributes": True} to work with SQLAlchemy ORM objects

Schema Design Pattern (same as Module 10):

- CalculationCreate → Input validation
- CalculationRead → Output serialization

Writing Tests — Unit and Integration

Module 11 had the most tests I have written so far—57 in total.

Unit Tests:

- Each operation class tested independently
- Factory tested to confirm it returns the correct class for each type
- CalculationCreate schema tested for valid inputs, invalid types, and divide-by-zero rejection

Integration Tests:

- Create calculations for all four operation types and verify stored results
- Confirm the database stores correct values by fetching back the record
- Confirm divide-by-zero returns 422
- Confirm invalid type returns 422
- Confirm non-existent user_id returns 404
- Confirm cascade delete removes calculations when a user is deleted

Key Lesson:

Testing the factory separately from the API was important. If a test fails at the integration level, I can quickly determine whether the issue is in schema validation, factory logic, or the database layer.

CI/CD Pipeline

The pipeline from Module 10 carried over with minimal changes:

1. Test Job

- Runs PostgreSQL 15 container
- Installs dependencies
- Runs all 57 tests

2. Build and Push Job

- Runs only if all tests pass
- Builds Docker image
- Pushes to kushyarwar/is601-module11:latest on Docker Hub
- Runs Trivy vulnerability scan

The only change from Module 10 was updating the Docker Hub image tag to is601-module11.

Challenges Faced

1. Model-level vs field-level validators

- Pydantic's @field_validator cannot access multiple fields at once
- Had to use @model_validator(mode="after") for the divide-by-zero check

2. Enum with SQLAlchemy

- SQLAlchemy stores the column as a plain String
- Pydantic handles the enum at the schema level
- Had to use calc.type.value when creating the model instance

3. Factory registry design

- Initially considered using if/elif chains but switched to a dictionary-based registry
- This makes the factory easier to extend and test

4. Test isolation with foreign keys

- Because calculations reference users, tests had to create a user first
- Table cleanup order mattered—calculations had to be cleared before users

What I Would Do Differently

If I redid this module, I would:

- Add a GET /users/{user_id}/calculations endpoint to retrieve user-specific calculations
- Store OperationType as a proper PostgreSQL ENUM column type instead of a plain string
- Add more edge cases to tests, such as very large numbers and negative operands

Conclusion

Module 11 taught me that good software design is not just about making things work—it is about making them easy to extend and maintain.

The factory pattern showed me how to isolate logic so that each piece of the application has a single responsibility. Pydantic's model validators demonstrated how to enforce business rules at the boundary, before data reaches the database.

Key concepts I will continue using:

- Enums for constrained string fields
- Factory pattern for extensible operation logic
- Model-level validators for cross-field validation
- Layered testing—unit tests for logic, integration tests for behavior

This module gave me a clearer picture of how real applications are structured, and how early design decisions can make testing and future development much easier.